CASE STUDY · CS-001

Market Intelligence & Research Agent.

A take-home brief for the **AI Engineer** role at Uniparticle. You'll design, build, and deploy a fully autonomous AI agent codenamed **M.I.R.A.** that monitors the equity markets, performs deep research on specific publicly traded companies, and generates structured, data-driven investment analysis reports.

What follows is the full brief: project overview, mandatory core requirements, advanced "spectacular outcome" features, recommended APIs, and submission deliverables. Read it end-to-end before you start scoping.

CODENAME	TIME LIMIT	HIGHLY VALUED	PAGES
M.I.R.A.	**48** hrs max	**< 24** hrs	**5** incl. cover

Market Intelligence & Research Agent (M.I.R.A.)

Case Study · AI Engineer

Time limit: 48 hours maximum · **Highly valued:** submission within 24 hours

1. Project Overview

The goal of this case study is to design, build, and deploy a fully functional, autonomous AI agent responsible for monitoring the equity markets, performing deep research on specific publicly traded companies, and generating highly structured, data-driven investment analysis reports.

The agent, M.I.R.A., must demonstrate advanced agentic behavior by autonomously deciding what research to perform, which tools to use, and how to synthesize diverse information sources (structured market data and unstructured news text) into a coherent, reliable report.

Why public equities: All required data prices, fundamentals, news, and peer comparisons is available through free, well-documented APIs. This lets us evaluate engineering quality without bottlenecking the assessment behind paid data subscriptions.

2. Core Requirements (Mandatory)

The solution must be built as a runnable, self-contained backend service (e.g., Python / FastAPI / Flask, Node.js / Express, or an equivalent framework).

A. Backend Architecture & Service Design

- **API Endpoint:** A primary REST endpoint `POST /analyze` that accepts a user query (e.g., "Analyze the near-term prospects of Tesla, Inc. (TSLA).").
- **Scalable Agent Core:** The analysis task must be delegated to a distinct Agent Service (or class/module) that runs asynchronously. The API endpoint must return a job ID immediately and expose `GET /status/{job_id}` for status checks.
- **Data Serialization:** All final output must be returned as a predictable, structured JSON object (Pydantic models, TypeScript interfaces, or equivalent JSON-schema definition).
- **Configuration:** Use environment variables or a configuration file to manage API keys and runtime settings (LLM provider, model name, polling intervals, etc.).

B. Agentic Behavior & Tool Use

The agent must exhibit multi-step, planning-oriented behavior (Chain-of-Thought or explicit planning) and utilize at least three distinct tools via function calling or an equivalent mechanism. The agent must use its planning ability to decide which tool to invoke next based on the input query and the results of previous tool calls.

- **Tool 1 Market Data Tool.** Fetches structured market data for the target ticker (price, daily change, volume, market cap, P/E, 52-week range, recent quarterly revenues) via Yahoo Finance (`yfinance`), Alpha Vantage, or Finnhub.
- **Tool 2 News Retriever & Sentiment Tool.** Fetches the 5 most recent, relevant news articles about the target company, then performs sentiment analysis returning a structured distribution (positive / negative / neutral) along with per-article sentiment tags. Sources: NewsAPI.org free tier. Sentiment may be performed by the LLM itself or by a local model (FinBERT, VADER) document the trade-off in the README.
- **Tool 3 Peer & Market Correlation Tool.** A function that simulates fetching structured financial data (e.g., a mock API call returning a company's last two quarterly revenue reports or recent stock price).

C. Output Structure

The final result stored and returned by `/status/{job_id}` must be a structured JSON report. Candidates define the schema, but it must include at minimum the following fields:

Field	Type	Description
`company_ticker`	String	The stock ticker analyzed (e.g., "TSLA").
`company_name`	String	Full registered name of the company.
`analysis_summary`	String	Concise paragraph synthesizing all findings.
`sentiment_score`	Float	Single derived score between -1.0 and 1.0.
`market_snapshot`	Object	Price, daily change, market cap, P/E, 52w range, last two quarterly revenues.
`correlation_analysis`	Object	Correlation coefficients vs. S&P 500, sector ETF, and peer(s).
`key_findings`	String[]	Top 3 actionable insights.
`tools_used`	String[]	Names of tools invoked, in order.
`citation_sources`	URL[]	All news article URLs referenced.
`generated_at`	ISO 8601	When the analysis completed.

3. Advanced Requirements (The "Spectacular Outcome")

Achieving these features demonstrates true mastery of AI engineering and scalable software design. This is where strong candidates differentiate themselves.

A. Dynamic Reflection and Self-Correction

The agent must implement a reflection loop. After receiving the initial tool results, the agent must **critique** (generate an internal thought process evaluating whether the gathered information is sufficient and unbiased) and, if needed, **re-plan** a second research pass before final synthesis. Concrete reflection triggers:

- If the stock's correlation with its sector ETF is above **0.95** over the analysis window, the stock is not moving on idiosyncratic news trigger a peer-comparison pass (fetch a direct competitor's recent news and price action) before drawing conclusions.
- If all news articles are over **72 hours old**, broaden the search or fetch alternative sources (e.g., recent SEC filings via EDGAR, analyst ratings) before concluding.
- If sentiment distribution is **perfectly neutral or evenly split**, fetch additional context (analyst commentary, earnings transcript snippets, or sector news) before issuing a final score.

B. Long-Term Memory (Persistent Monitoring)

- **Endpoint:** A trigger endpoint `POST /monitor_start` that registers a background task to monitor a specific ticker on a configurable cadence (default: every 24 hours during trading days).
- **Trigger Logic:** The monitoring task runs M.I.R.A.'s core analysis only if at least one of the following is true since the last run: (a) 5 or more new articles have been published, OR (b) the closing price has deviated by more than 2 standard deviations from its 30-day mean, OR (c) trading volume has spiked above 2× the 30-day average.
- **Notification:** When criteria are met, the agent generates a new analysis and stores it, marking it with the tag `PROACTIVE_ALERT` and recording which trigger fired.
- **State:** Per-ticker state (last run timestamp, baseline price/volume stats, last article IDs seen) must be persisted so monitoring survives container restarts.

C. Observability & Cost Controls

Production agents must be inspectable and economically bounded. The solution must include:

- Per-job structured logs capturing each tool invocation, its inputs, latency, and success/failure status.
- Token usage tracking per job (prompt tokens, completion tokens, and an estimated cost based on the configured model).
- A configurable maximum tool-call budget per job (default: 10) to prevent runaway agent loops.

D. Evaluation

- At least 3 documented test cases with input queries and expected output characteristics (e.g., "AAPL analysis should always include correlation with itself = 1.0", "unknown ticker should return a graceful error, not a hallucinated report", "a query about a delisted ticker should fail gracefully").
- A short written discussion (½ 1 page in the README) of how the candidate would measure agent quality at scale: ground-truth comparisons, LLM-as-judge, regression suites, or any thoughtful approach.

E. Containerization and Deployment Readiness

- **Docker:** A working Dockerfile that packages the application and its dependencies. The application must be runnable via a single `docker build` followed by `docker run` command.
- `docker-compose.yml` is encouraged if multiple services are involved (e.g., API + worker + Redis).
- **Documentation:** A README section explaining how to execute the service, hit the primary API endpoint, and check job status.

4. Recommended APIs (All Free, All Accessible)

All required functionality can be implemented using the following fully accessible data sources. Candidates are free to substitute equivalents, but should not need any paid subscription to complete the assessment.

API / Library	Use Case	Auth	Free Tier
`yfinance` (Yahoo Finance)	Prices, fundamentals, OHLC, peer & index data	None	Effectively unlimited
Alpha Vantage	Prices, fundamentals, technical indicators	Free key	5 req/min, 500/day
Finnhub	Prices, company news, fundamentals	Free key	60 req/min
NewsAPI.org	General news search by company name	Free key	100 req/day
Marketaux	Financial news with sentiment tags	Free key	100 req/day
SEC EDGAR	Filings (10-K, 10-Q, 8-K) for reflection passes	None	10 req/sec

LLM provider: Any provider with function-calling support is acceptable (Anthropic Claude, OpenAI GPT-4, Google Gemini, or open-weight models via Groq / Together). The candidate should document their choice and rationale.

5. Deliverables

Candidates must submit a compressed archive (`.zip` or `.tar.gz`) containing:

- **Source Code:** The complete, runnable backend service, including the agent logic, tool implementations, and persistence layer.
- **Dockerfile** (and optionally `docker-compose.yml`): The container definition.
- **README.md:** Architectural overview (with a simple diagram), technology choices and rationale, setup and run instructions, the evaluation discussion described in section 3.D, and a clearly marked "Known Limitations" section.
- **Dependency manifest:** `requirements.txt` , `pyproject.toml` , `package.json` , or equivalent.
- **.env.example:** Listing all required environment variables (without secrets).
- **Sample output:** At least one full JSON report produced by the agent, included as `sample_output.json` .
- **Postman collection:** A `postman_collection.json` exporting all API endpoints (`/analyze` , `/status/{job_id}` , etc.) with example requests and responses for quick reviewer testing.